



241-405 Optimization

Graph Optimization1

Introduction

YOSSAWEE KEAOMANEE



Graph Basics & Representation

วัตถุประสงค์ (Objectives)

- ทำความเข้าใจนิยามของกราฟ (Graph) ประโยชน์ในการแก้ปัญหาคำตอบที่เหมาะสมที่สุด (Optimization)
- การแทนกราฟในรูปแบบต่าง ๆ สำหรับนำไปใช้ในอัลกอริทึม (Algorithms)
- สร้างพื้นฐานความรู้สำหรับหัวข้อที่เกี่ยวข้อง เช่น ปัญหาเส้นทางสั้นที่สุด (Shortest Path) และต้นไม้ครอบคลุมขั้นต่ำ (Minimum Spanning Tree: MST).



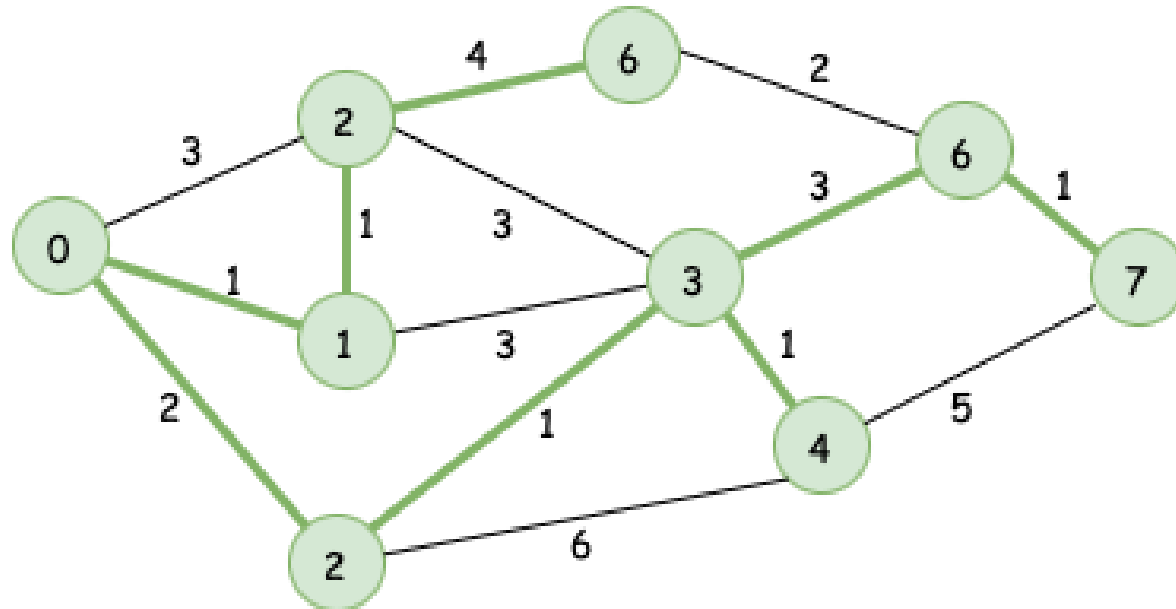
Outline

- นิยามของกราฟ
- การใช้กราฟในการหาค่าเหมาะสม (Optimization)
- ตัวอย่างปัญหา
- การแทนกราฟในรูปแบบต่าง ๆ



Definition of a Graph

โครงสร้างทางคณิตศาสตร์ (Mathematical Structure) ที่ใช้กำหนด ขอบเขตของปัญหา (Search Space/Feasible Regions) และ เงื่อนไขบังคับ (Constraints) เพื่อหาเส้นทาง, ค่า, หรือรูปแบบการเชื่อมต่อที่ให้ผลลัพธ์ที่ดีที่สุด (เช่น ค่าใช้จ่ายน้อยที่สุด หรือประสิทธิภาพสูงสุด)





Key Graph Components

- จุดยอด (Vertices - V): ปมหรือโหนดต่าง ๆ ในกราฟ (เช่น เมือง, เครื่องเซิร์ฟเวอร์)
- เส้นเชื่อม (Edges - E): เส้นที่เชื่อมต่อระหว่างจุดยอดแต่ละจุด (เช่น ถนน, สายเคเบิล)
- ดีกรี (Degree): จำนวนเส้นเชื่อมที่ต่ออยู่กับจุดยอดนั้น ๆ
- วิถี, วัฏจักร, และกราฟเชื่อมโยง (Paths, Cycles, Connected Graphs): แนวคิดพื้นฐานที่ใช้สำหรับการค้นหาเส้นทางหรือการนำทางในกราฟ



Definition of a Graph

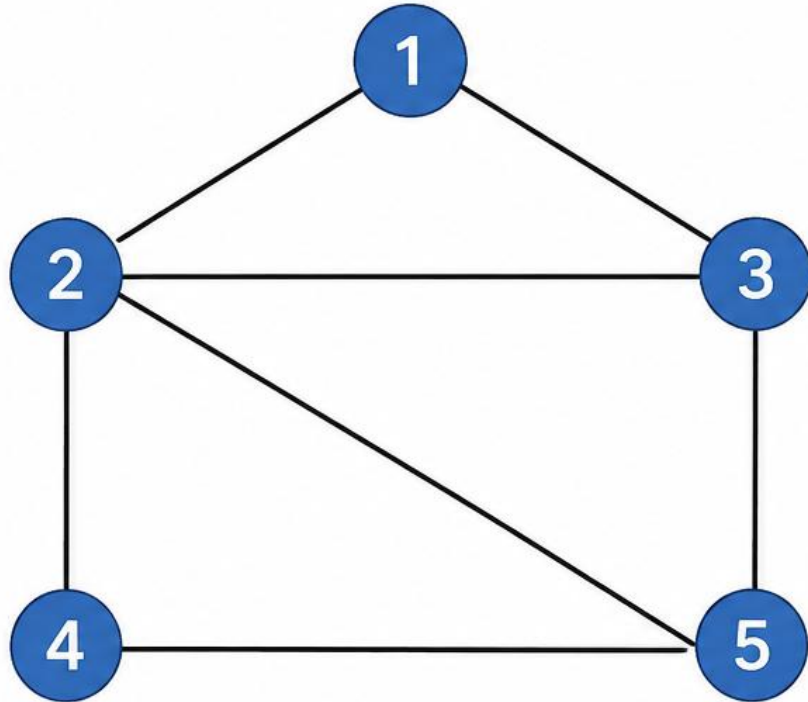
ประเภทของกราฟ (Types of graphs) พื้นฐาน:

Type	Description
กราฟไม่มีทิศทาง (Undirected Graph)	เส้นเชื่อม (Edges) ไม่ระบุทิศทาง สามารถเดินทางไปกลับระหว่างจุดยอดได้อย่างอิสระ
กราฟมีทิศทาง/ไดกราฟ (Directed Graph / Digraph)	เส้นเชื่อมมีทิศทางที่ชัดเจน (แสดงด้วยลูกศร) บังคับทิศทางการเดินทางจากจุดหนึ่งไปยังอีกจุดหนึ่ง
กราฟถ่วงน้ำหนัก (Weighted Graph)	เส้นเชื่อมมีค่าน้ำหนักกำกับอยู่ เพื่อบอกข้อมูลเฉพาะตัว เช่น ระยะทาง, ต้นทุน, หรือเวลา
กราฟไม่ถ่วงน้ำหนัก (Unweighted Graph)	เส้นเชื่อมไม่มีค่าน้ำหนักกำกับ ทุกเส้นเชื่อมมีความสำคัญเท่ากันหมด



กราฟไม่มีทิศทาง (Undirected Graph)

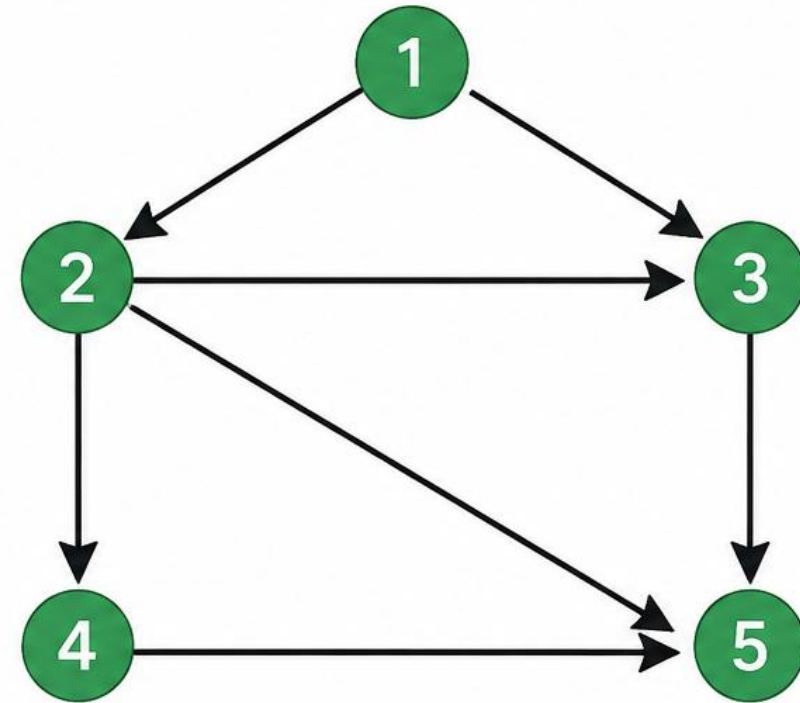
เส้นเชื่อมหรือเอจ (Edge) ไม่มีทิศทาง
การเชื่อมจากจุด A ไปยังจุด B
ถือเป็นการเชื่อมเดียวกับจาก B ไปยัง A



เช่น 1-2 คือการเชื่อมเดียวกับ 2-1

กราฟมีทิศทาง / ไดกราฟ (Directed Graph / Digraph)

เส้นเชื่อมหรืออาร์ก (Arc) มีทิศทาง
การเชื่อมจากจุด A ไปยังจุด B
ไม่เหมือนกับการเชื่อมจาก B ไปยัง A

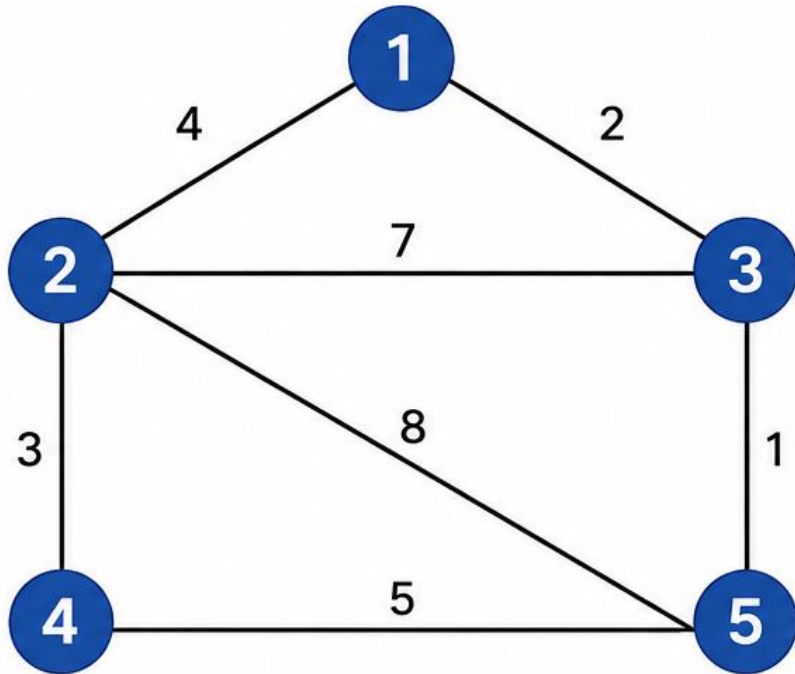


เช่น $1 \rightarrow 2$ ไม่เหมือนกับ $2 \rightarrow 1$



กราฟถ่วงน้ำหนัก (Weighted Graph)

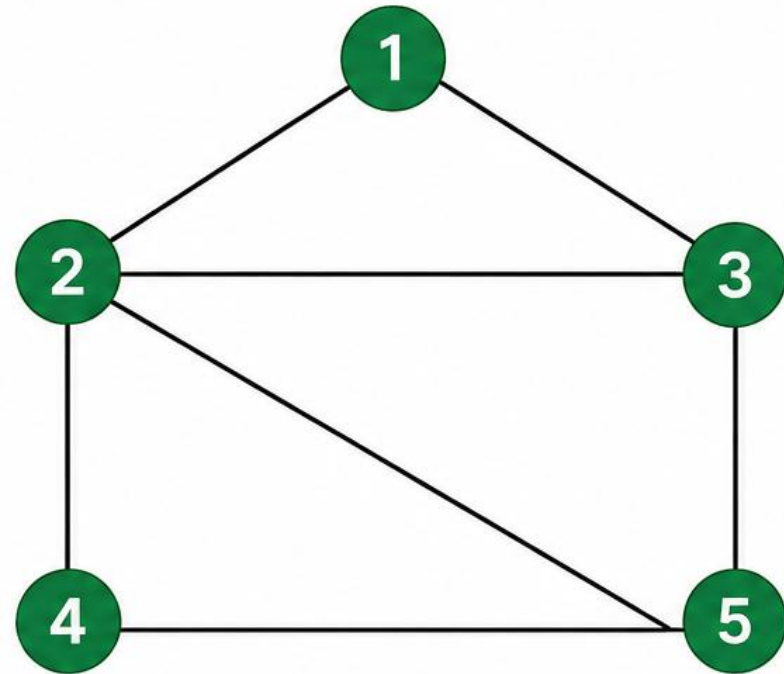
กราฟที่เส้นเชื่อมหรืออาร์กแต่ละเส้น
มีค่าน้ำหนักกำกับ (Weight)
ซึ่งอาจแทนระยะทาง, เวลา, ค่าใช้จ่าย ฯลฯ



เช่น เส้นเชื่อม 1-2 มีน้ำหนัก 4
เส้นเชื่อม 2-5 มีน้ำหนัก 8

กราฟไม่ถ่วงน้ำหนัก (Unweighted Graph)

กราฟที่เส้นเชื่อมหรืออาร์กแต่ละเส้น
ไม่มีค่าน้ำหนักกำกับ
ถือว่าทุกเส้นมีค่าเท่ากัน (มักกำหนดเป็น 1)



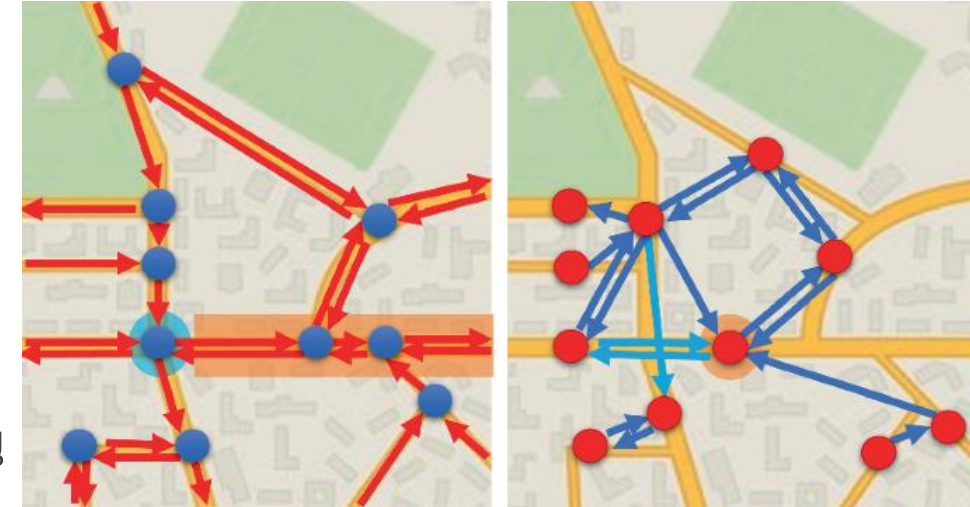
เช่น ทุกเส้นเชื่อมมีน้ำหนัก = 1
เส้นเชื่อม 1-2 = 1 เช่นเดียวกับ 2-1



Why we use the graph in optimization?

การสร้างแบบจำลองเครือข่ายในโลกจริง (Modeling Real - World Networks) สามารถใช้ กราฟ (Graphs) เป็นตัวแทนของระบบต่าง ๆ ได้อย่างหลากหลาย ดังนี้:

- เครือข่ายการขนส่ง: เช่น เมือง เส้นทางถนน
- เครือข่ายการสื่อสาร: เช่น อุปกรณ์และการเชื่อมต่อสัญญาณ
- ห่วงโซ่อุปทาน (Supply Chains): เช่น คลังสินค้าและเส้นทางการจัดส่ง
- ลำดับงานในโครงการ: เช่น กิจกรรมและความสัมพันธ์ที่ต้องพึ่งพากัน



Bachechi, C., Po, L. (2022). Road Network Graph Representation for Traffic Analysis and Routing. In: Chiusano, S., Cerquitelli, T., Wrembel, R. (eds) Advances in Databases and Information Systems. ADBIS 2022. Lecture Notes in Computer Science, vol 13389.

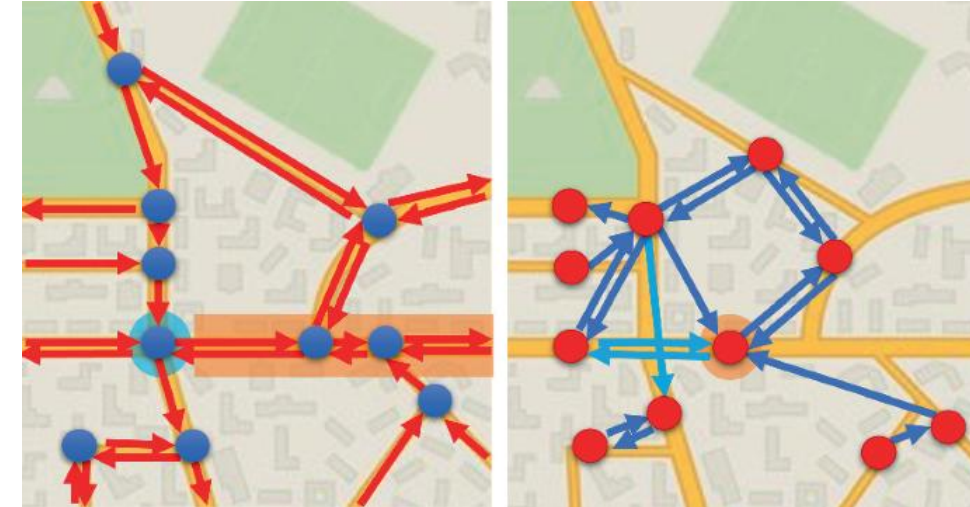


Why we use the graph in optimization?

การแทนความสัมพันธ์อย่างมีประสิทธิภาพ (Efficient Representation of Relationships)

โครงสร้างข้อมูลแบบกราฟ:

- ใช้โหนด (Nodes / Vertices): แทนสิ่งต่าง ๆ หรือหน่วยข้อมูล (เช่น เมือง, เครื่องจักร, ชิ้นงาน)
- ใช้เส้นเชื่อม (Edges): แทนความสัมพันธ์ระหว่างสิ่งเหล่านั้น (เช่น ระยะทาง, ค่าใช้จ่าย, เวลา)



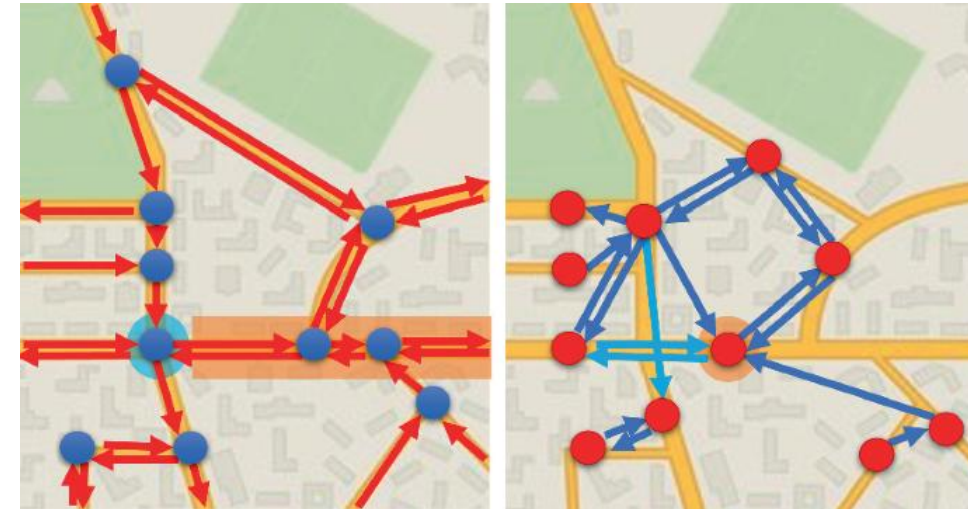
Bachechi, C., Po, L. (2022). Road Network Graph Representation for Traffic Analysis and Routing. In: Chiusano, S., Cerquitelli, T., Wrembel, R. (eds) Advances in Databases and Information Systems. ADBIS 2022. Lecture Notes in Computer Science, vol 13389.



Why we use the graph in optimization?

เมื่อสร้างแบบจำลองแล้ว การหาความคุ้มค่าสูงสุด (Optimization) จะมุ่งเน้นไปที่การค้นหาด้านต่างๆ เช่น:

- เส้นทางที่สั้นที่สุด (The shortest path)
- ต้นทุนที่ต่ำที่สุด (The minimum cost)
- ตารางเวลาที่เหมาะสมที่สุด (The optimal schedule)
- ปริมาณการไหลสูงสุด (The maximum flow)



Bachechi, C., Po, L. (2022). Road Network Graph Representation for Traffic Analysis and Routing. In: Chiusano, S., Cerquitelli, T., Wrembel, R. (eds) Advances in Databases and Information Systems. ADBIS 2022. Lecture Notes in Computer Science, vol 13389.



Why we use the graph in optimization?



○ Nodes (จุดยอด): ได้แก่ Person (บุคคล), Communication Methods (ช่องทางสื่อสาร) และ Topic (หัวข้อสนทนา)

○ Edges (เส้นเชื่อม):

- Directed: ลูกศรสีส้ม แสดงทิศทางการสื่อสาร
- Undirected: เส้นตรง แสดงความสัมพันธ์ระหว่างบุคคล

○ Weights (น้ำหนัก): ตัวเลขบนเส้น (0.2–1.0) แทนความถี่ หรือความน่าจะเป็นในการสื่อสาร



Why we use the graph in optimization?

โจทย์ปัญหาการหาค่าเหมาะสมที่สุดจำนวนมากมีโครงสร้างพื้นฐานที่อิงตามทฤษฎีกราฟโดยธรรมชาติ

โจทย์ปัญหาการหาค่าเหมาะสมที่สุด	การประยุกต์ใช้ทฤษฎีกราฟ(Graph Use)
• การหาเส้นทางสั้นที่สุด(Shortest Path)	• อัลกอริทึมของ Dijkstra, Bellman-Ford, และ A*
• ต้นไม้แบบทอดข้ามที่น้อยที่สุด(Minimum Spanning Tree)	• อัลกอริทึมของ Kruskal และ Prim
• ปัญหานักเดินทางขายของ(Traveling Salesman)	• การหาค่าเหมาะสมที่สุดด้วยวัฏจักรแฮมิลโทเนียนบนกราฟถ่วงน้ำหนัก (Hamiltonian cycle optimization on weighted graph)
• อัตราการไหลสูงสุด / รอยตัดต่ำสุด(Maximum Flow / Minimum Cut)	• อัลกอริทึมการไหลในเครือข่าย เช่น Ford-Fulkerson เป็นต้น(Network flow algorithms)
• การจัดตารางงาน(Job Scheduling)	• การใช้กราฟระบุทิศทางแบบไม่มีวัฏจักร (DAGs) และการเรียงลำดับเชิงทอพอโลยี(Topological sorting)



Why we use the graph in optimization?

- ความเกี่ยวเนื่องสัมพันธ์กันระหว่างตัวแปร (Dependencies between variables): แสดงให้เห็นว่าตัวแปรใดต้องขึ้นตรงหรือรอข้อมูลจากตัวแปรใดก่อน
- ความขัดแย้งหรือข้อจำกัดในการใช้ร่วมกัน (Conflicts or exclusions): แสดงสิ่งที่ไม่สามารถเกิดขึ้นพร้อมกันได้ หรือเงื่อนไขที่ขัดแย้งกัน (เช่น งานสองงานที่ใช้ทรัพยากรเดียวกันพร้อมกันไม่ได้)
- ข้อจำกัดด้านอัตราการไหล (Flow limitations): แสดงขีดความสามารถสูงสุดหรือปริมาณที่ยอมรับได้ในแต่ละเส้นทาง (เช่น ปริมาณข้อมูลในสายส่ง หรือปริมาณน้ำในท่อ)



Why we use the graph in optimization?

ความยืดหยุ่นของอัลกอริทึม: เหมาะสำหรับใช้อธิบายและแก้ปัญหาด้วยอัลกอริทึมการค้นหาและการหาค่าเหมาะสมที่สุดที่หลากหลาย เช่น:

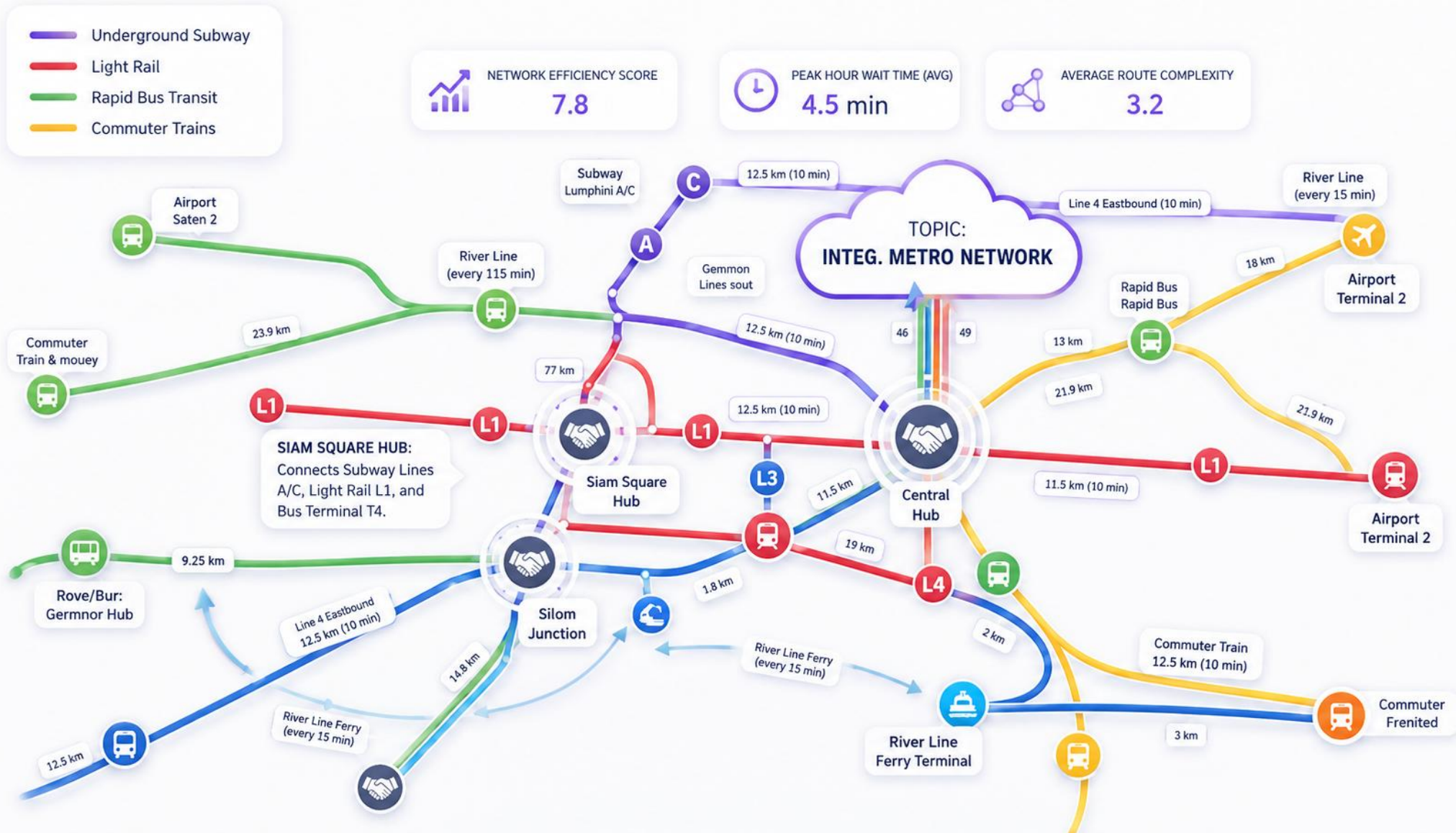
- **วิธีฮิวริสติกและเมทาฮิวริสติก (Heuristics & Metaheuristics):** เช่น อัลกอริทึมเชิงพันธุกรรมบนโครงสร้างกราฟ (Genetic Algorithms on graphs) เพื่อหาคำตอบที่ดีที่สุดในการแก้ปัญหาที่ซับซ้อน
- **การโปรแกรมเชิงพลวัต (Dynamic Programming):** เช่น อัลกอริทึม Floyd-Warshall (ใช้หาเส้นทางสั้นที่สุดระหว่างทุกคู่จุด)
- **อัลกอริทึมแบบละโมภ (Greedy Algorithms):** เช่น การเลือกทางเลือกที่ดีที่สุดในแต่ละก้าวทันที เช่น อัลกอริทึมของ Prim หรือ Kruskal



Example problem

การแปลงเงื่อนไขให้เป็นแบบจำลองกราฟ (Constraint Encoding) เช่น
แบบจำลองระบบขนส่งมวลชนในเมืองให้อยู่ในรูปของกราฟ:

- จุดยอด (Vertices / Nodes) = สถานี
- เส้นเชื่อม (Edges) = เส้นทางการเดินทางระหว่างสถานี
- น้ำหนัก (Weights) = เวลาที่ใช้ในการเดินทาง





Graph Representations

เมทริกซ์ประชิด (Adjacency Matrix) เมทริกซ์ (ตาราง) ที่มีขนาด $|V| \times |V|$.

- **Pros:** สามารถตรวจสอบการเชื่อมต่อของเส้นเชื่อม (Edge lookup) ได้อย่างรวดเร็ว โดยใช้เวลาคงที่
- **Cons:** สิ้นเปลืองพื้นที่ในหน่วยความจำสูงสำหรับกราฟที่มีขนาดใหญ่ (มีจำนวนจุดยอดมาก).

ตัวอย่าง:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$



Graph Representations

รายการอุบัติการณ์ (Adjacency List)

สำหรับแต่ละจุดยอด (Vertex) จะมีการเก็บรวบรวมรายชื่อของจุดยอดอื่นๆ ที่มีเส้นเชื่อมต่อกันเอาไว้

■ **ข้อดี:** ประหยัดพื้นที่จัดเก็บข้อมูลสูง สำหรับกราฟแบบเบาบาง (Sparse Graphs หรือกราฟที่มีจำนวนเส้นเชื่อมค่อนข้างน้อย)

■ **ข้อเสีย:** ใช้เวลาในการตรวจสอบช้ากว่า ว่ามีเส้นเชื่อม (Edge) ระหว่างจุดยอดสองจุดที่ต้องการหรือไม่.

A: B, D

B: A, C

C: B, D

D: A, C



Graph Representations

ตัวอย่างการสร้างแบบจำลองกราฟสำหรับ 4 เมือง

จากข้อมูล มีเมือง 4 เมือง (A, B, C, D) ที่เชื่อมต่อกันแบบสองทิศทาง ($\leftrightarrow$) พร้อมระยะทาง (น้ำหนักของเส้นเชื่อม) ดังนี้:

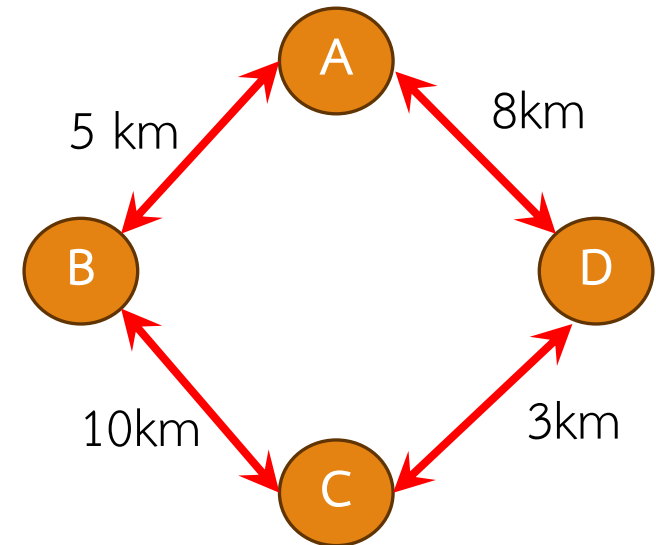
A $\leftrightarrow$ B (5 km)

B $\leftrightarrow$ C (10 km)

C $\leftrightarrow$ D (3 km)

A $\leftrightarrow$ D (8 km)

เขียนทั้งเมทริกซ์ประชิด (Adjacency Matrix) และรายการประชิด (Adjacency List) ของกราฟนี้

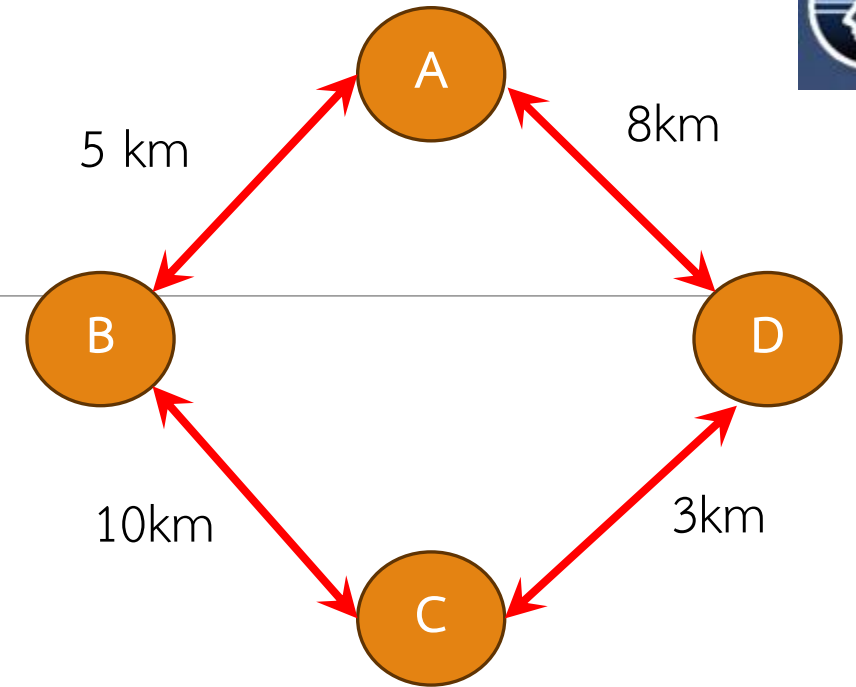




Graph Representations

เมทริกซ์ประชิด (Adjacency Matrix)

- เป็นตารางขนาด 4×4 โดยค่าในแถวที่ i และคอลัมน์ที่ j คือระยะทางระหว่างเมือง i และเมือง j
- ค่า 0 จะหมายถึงไม่มีเส้นทางเชื่อมต่อกันโดยตรง (หรืออาจใช้สัญลักษณ์ ∞ หากต้องการสื่อว่าเมืองเหล่านั้นไม่สามารถเดินทางไปถึงกันได้ แต่ในที่นี้จะใช้ 0 เพื่อความง่ายไม่ซับซ้อน)

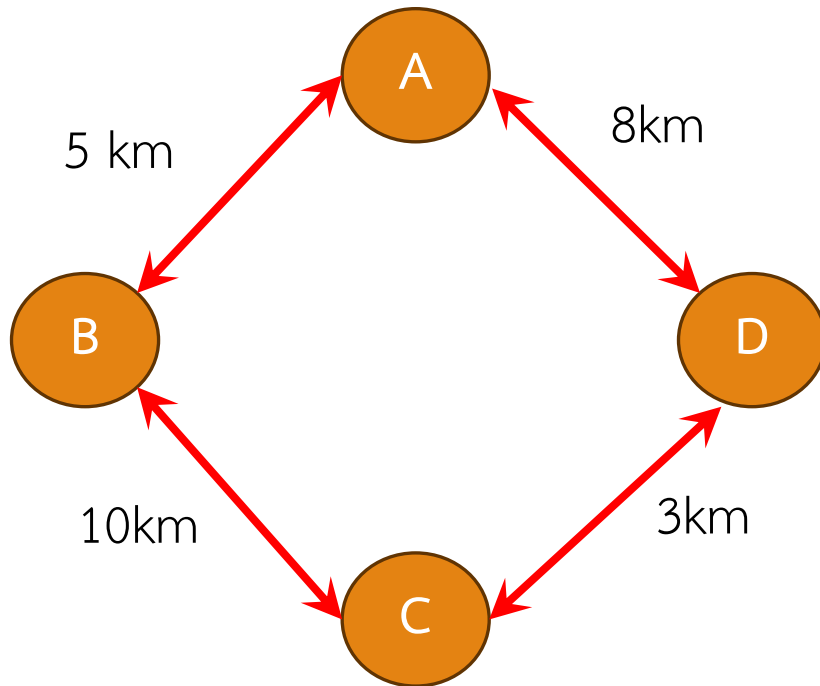


	A	B	C	D
A	0	5	0	8
B	5	0	10	0
C	0	10	0	5
D	8	0	5	0



Graph Representations

รายการส่วนประชิด (Adjacency List) แสดงรายชื่อเมืองที่อยู่ติดกัน (เมืองข้างเคียง) ของแต่ละเมือง พร้อมระยะระยะทางที่สอดคล้องกัน



A: [(B, 5), (D, 8)]

B: [(A, 5), (C, 10)]

C: [(B, 10), (D, 3)]

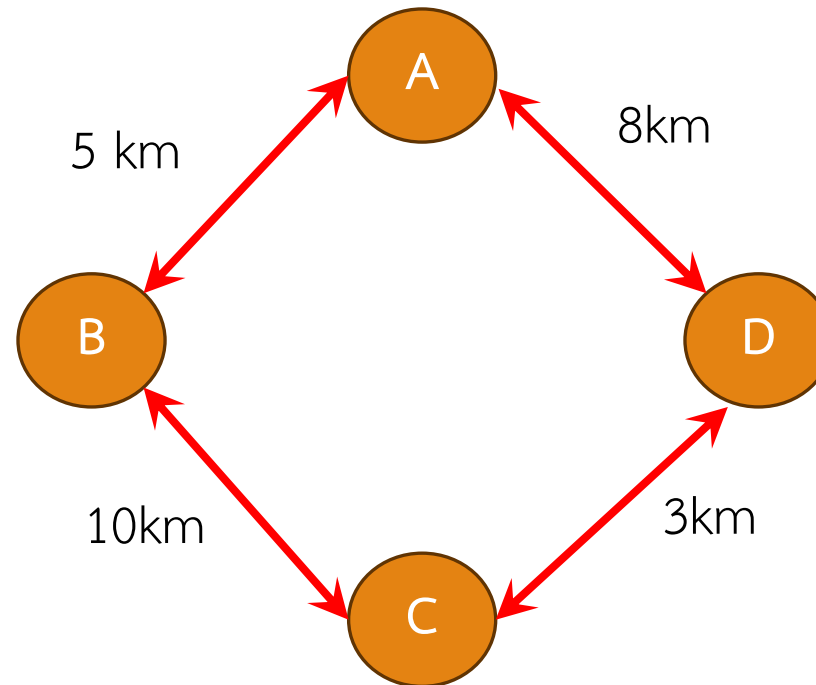
D: [(A, 8), (C, 3)]



Graph Representations

Edge List เก็บเป็นรายการของ Edge

- (A,B)
- (A,D)
- (B,C)
- (C,D)

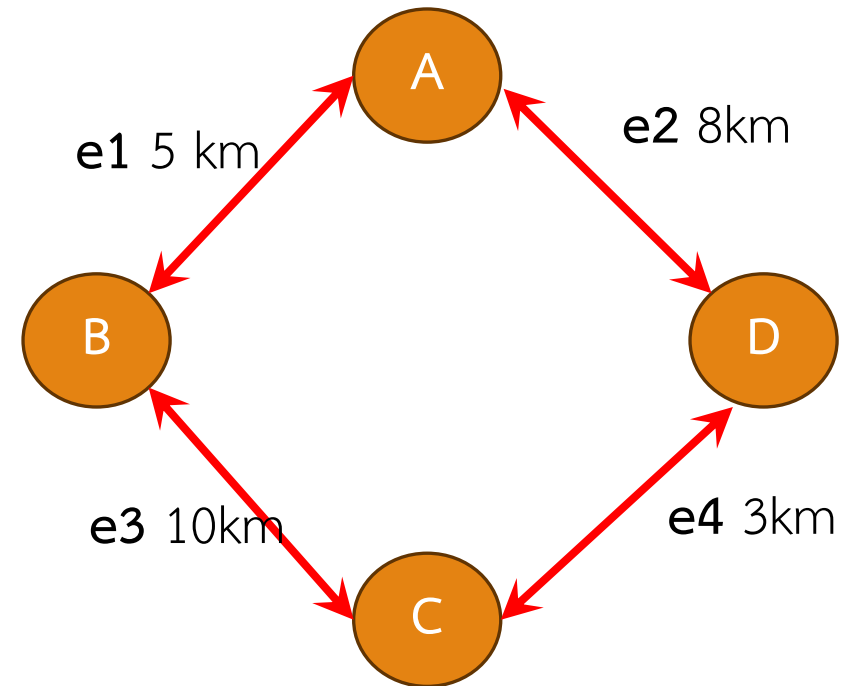




Graph Representations

Incidence Matrix ใช้แสดงว่า Node ไດ
เกี่ยวข้องกับ Edge ไດ

	e1	e2	e3	e4
A	1	1	0	0
B	1	0	1	0
C	0	0	1	1
D	0	1	0	1





Applications of Graph Representations

การประยุกต์ใช้งาน(Application)	รูปแบบการแทนข้อมูลที่แนะนำ (Recommended Representation)
• เครือข่ายสังคมออนไลน์(Social Networks) • แผนที่เมือง / ระบบนำทาง(City Maps / Navigation) • การหาเส้นทางในเครือข่ายอินเทอร์เน็ต (Network Routing / Internet)	• รายการส่วนประชิด(Adjacency List) • รายการส่วนประชิดแบบถ่วงน้ำหนัก (Weighted Adjacency List) • เมทริกซ์ส่วนประชิด(Adjacency Matrix)